Linear Regression — Complete Learning Notes

Dataset: Ames Housing (Kaggle) | Problem Type: Regression (Target = SalePrice)

Full Roadmap (Reference)

- Phase 1 — Data Inspection
- Phase 2 — Target Variable Analysis (skewness, normality)
- Phase 3 — Numerical Features EDA + Statistical Inference
- Phase 4 — Multicollinearity Check (VIF)
- Phase 5 — Categorical Features EDA
- Phase 6 — Missing Value Handling
- Phase 7 — Feature Engineering
- Phase 8 — Train-Test Split (done before EDA/FE on full data to avoid leakage)
- Phase 9 — Scratch Implementation (Normal Equation + Gradient Descent)
- Phase 10 — sklearn Implementation
- Phase 11 — Evaluation + Statistical Inference (R^2, Adjusted R^2, RMSE, MAE, residual diagnostics, OLS F-test/t-test)
- Phase 12 — Comparison & Conclusion

Corrected order: Phase 1, 2 (raw inspection) → Phase 8 (train-test split) → Phase 3, 4, 5 (EDA only on train set) → Phase 6, 7 (cleaning + FE, fit on train, transform test) → Phase 9, 10 (train models) → Phase 11, 12 (evaluate and compare).

PHASE 1: Data Inspection

Why This Phase Exists

Before training any model, we need to understand the 'personality' of the dataset. This phase does not discover final insights by itself — it generates the QUESTIONS that later phases (EDA, FE, cleaning) will answer. It tells us where to focus effort next.

Step 1: Shape (rows x columns)

What: df.shape after dropping the Id column (Id is just a row identifier, not predictive).

Why it matters:

- Tells us if we have enough data relative to the number of features. Few rows + many columns = risk of 'curse of dimensionality' and overfitting → model may need stronger regularization.
- Tells us how reliable a single train-test split will be. Small datasets (~1460 rows here) may need cross-validation for a more reliable evaluation instead of trusting a single split.

Step 2: Data Types Breakdown

What: df.dtypes.value_counts() — count of numeric vs categorical (object) columns.

Why it matters:

- Numeric and categorical columns need completely different treatment later: numeric columns need distribution checks, scaling, correlation analysis; categorical columns need encoding decisions (ordinal vs nominal).
- If a large portion of columns are categorical (as in Ames — 43 categorical columns), ignoring them and only doing numeric EDA would miss major predictors like 'Neighborhood'.

Step 3: Missing Values Check

What: count and % of missing values per column, sorted descending.

This is the most important step in Phase 1. A missing value is not just a 'cleaning problem' — it tells a story about WHY the value is absent, and that story determines how we handle it later (Phase 6).

- Example: PoolQC (Pool Quality) missing in ~99.5% of rows does NOT mean bad data collection. It means 99.5% of houses simply do not have a pool. The missing value itself is information ('no pool'), not an error.
- Example: LotFrontage missing in ~17.7% of rows could be genuinely random missingness, or could be systematic for certain property types.
- This distinction (relates to MCAR/MAR/MNAR from statistics) decides the imputation strategy: 'meaningful NA' columns get filled with 'None'/0 (preserves information), genuinely missing values get mean/median/mode or model-based imputation.
- Mistake to avoid: blindly filling every categorical column with mode (most frequent value) would assign a fake pool quality to houses that have no pool at all — corrupting the signal the model would otherwise learn.

Step 4: Target Variable (SalePrice) Describe

What: df['SalePrice'].describe() — mean, median, std, min, max, quartiles.

- Range check: Min ~34,900 vs Max ~755,000 (~20x difference) signals potential outliers — need to verify whether extreme values are genuine mansions or data-entry errors.
- Mean vs Median comparison: if mean > median, this is an early signal of right-skew (a few very expensive houses pull the mean up). This is a first hint — to be formally confirmed in Phase 2 with skewness/Shapiro-Wilk test.

Key Takeaway

Phase 1 does not give final answers — it gives the right QUESTIONS for later phases: 'How much missing data, and what does it mean?' (answered in Phase 6), 'Is the target skewed?' (confirmed in Phase 2), 'Which features are numeric vs categorical?' (explored in Phase 3/5).

PHASE 2: Target Variable Analysis (SalePrice)

Why This Phase Exists

Phase 1 gave a hint: mean (180,921) > median (163,000), suggesting skew. Phase 2 formally confirms this and decides whether the target needs a transformation. This matters because Linear Regression's Assumption 4 (normality of residuals) depends heavily on the target's distribution — if the target is

heavily skewed, residuals will likely be skewed too, making confidence intervals and p-values (used later in Phase 11) unreliable.

Step 1: Skewness and Kurtosis

Skewness measures symmetry (0 = symmetric/normal-like; positive = long right tail; negative = long left tail). Kurtosis measures 'peakedness' and how heavy the tails are (high kurtosis = more extreme outliers than normal). Rule of thumb: |skewness| > 1 is considered 'highly skewed' and typically needs transformation.

Step 2: Visualize — Histogram + KDE

A histogram with a KDE curve makes the skew visually obvious — the long right tail (expensive houses pulling the distribution) is immediately visible compared to a normal bell curve.

Step 3: Formal Normality Test — Shapiro-Wilk

H0: data comes from a normal distribution. H1: data does not. A very small p-value (< 0.05) rejects H0, confirming non-normality statistically. Caveat: Shapiro-Wilk is very sensitive with large samples (1000+ rows) — small deviations can appear 'significant', so always cross-check with the skewness value and the plot, not just the p-value.

Step 4: Log-Transform and Re-check

Apply np.log1p(SalePrice) and recompute skewness + Shapiro-Wilk. If skewness drops significantly (closer to 0) and the distribution visually looks bell-shaped, log-transform is the right call.

Result on this dataset:

Raw SalePrice was heavily right-skewed (skewness ~1.88). After applying log1p transform, the distribution became roughly normal-shaped (skewness dropped close to 0). Decision: train the model on log(SalePrice) as the target, and convert predictions back to the original rupee scale using np.expm1() before computing evaluation metrics like RMSE/MAE (so error metrics stay interpretable in real currency, not in log-scale).

Doubt Clarified: Why Are Real Estate Prices Multiplicative, Not Additive?

Additive pattern: a change is a fixed absolute amount regardless of the base value (e.g., temperature +5 degrees means the same thing whether it's 20°C or 100°C). Linear Regression's formula (y = w1x1 + b) naturally models additive effects.

Multiplicative pattern: a change is a fixed PERCENTAGE of the base value, so the absolute size of the change depends on the base. Example: a location improvement that raises prices by 10% moves a Rs 10 lakh house to Rs 11 lakh (+Rs 1 lakh) but moves a Rs 1 crore house to Rs 1.1 crore (+Rs 10 lakh). Same percentage effect, very different absolute effect. This happens because house price is roughly a PRODUCT of factors: Price ~ Base x Location x Quality x Size — when one multiplier changes, the whole product scales, not shifts by a fixed amount.

Why this breaks Linear Regression: the model tries to find one fixed slope (fixed rupee effect per unit of a feature) that works for both cheap and expensive houses — but no single fixed number can correctly describe both a Rs 1 lakh change and a Rs 10 lakh change for 'the same' underlying effect. This leaves

larger errors for expensive houses, creating heteroscedasticity (violating Assumption 3: constant error variance).

How log fixes it: log(a x b) = log(a) + log(b) — logarithm converts multiplication into addition. So if Price = Base x Location x Quality x Size, then log(Price) = log(Base) + log(Location) + log(Quality) + log(Size) — now all factors ADD, which is exactly the form Linear Regression expects. Numeric check: log(11)-log(10) = 0.095 and log(110)-log(100) = 0.095 — identical on the log scale, even though the absolute rupee changes were very different. This is also why the raw distribution is right-skewed: log compresses large values more than small values, pulling in the long right tail and making the distribution roughly symmetric.

PHASE 3: Numerical Features EDA + Statistical Inference

Why This Phase Exists

So far we only understood the target. Now we need to understand the RELATIONSHIP between features and the target — which numeric features genuinely help predict SalePrice, and which are noise. This phase directly tests Assumption 1 (Linearity), the foundation of the model we are about to build.

Step 1: Scatter Plots — Top Numeric Features vs SalePrice

Plot key numeric features (GrLivArea, OverallQual, TotalBsmtSF, GarageArea, YearBuilt, LotArea, etc.) against SalePrice. This is a visual linearity check — a straight-line trend is a good sign for Linear Regression; a curved pattern (exponential, U-shape) means the relationship needs a transformation (log, polynomial feature) before a linear model can capture it well.

Step 2: Pearson Correlation + Hypothesis Testing

Compute df.corr()['SalePrice'] for all numeric features, and additionally run scipy.stats.pearsonr(x, y) on important features to get a p-value alongside the correlation coefficient (r).

Intuition: r tells us the STRENGTH and DIRECTION of a relationship (-1 to +1). But r alone does not tell us whether that relationship is genuinely real in the population, or just a random fluke of this particular 1460-house sample. The hypothesis test resolves this: H0 = 'true correlation is 0 in the population' (no real relationship, whatever we see in the sample is luck), H1 = 'true correlation is not 0' (the relationship is real). A very small p-value means: if H0 were actually true, seeing a correlation this strong by pure chance would be extremely unlikely — so we reject H0 and conclude the relationship is genuine.

Nuance: with a large sample (1460 rows), even a small correlation (like 0.1) can come out 'statistically significant' (small p-value), but may still be practically too weak to matter. Always look at BOTH the magnitude of r (practical importance) and the p-value (statistical significance), not just one.

Step 3: Correlation Heatmap — Numeric Features Among Themselves

Build a correlation matrix of all numeric features against each other and visualize with a heatmap. This is the first visual signal of multicollinearity (Assumption 2), to be formally confirmed with VIF in Phase 4. Feature pairs with very high correlation (e.g. 0.8+, like GarageCars and GarageArea) are a red flag that they carry redundant information.

Doubt Clarified: Population vs Sample, and What a p-value Actually Means

Population = the entire real-world set of house sales (e.g., all houses ever sold in Ames). Sample = the 1460 houses actually available in the dataset, a small piece of that population. The core question: if we compute r = 0.71 between GrLivArea and SalePrice in our 1460-house sample, can we be confident this relationship genuinely exists in the full population — or could it just be a coincidence of this particular random sample?

Two competing stories: H0 ('boring' story) = there is no real relationship in the population (true correlation = 0); whatever correlation we see in the sample is purely due to random sampling luck. H1 ('interesting' story) = the relationship is genuinely real, and the sample correlation reflects it. The p-value literally answers: 'If H0 were true (no real relationship), what is the probability of seeing a correlation this large or larger, purely by chance, given our sample size?'

If that probability is tiny (very small p-value), it becomes hard to believe 'this happened by pure luck' — so we reject H0 and conclude the relationship is genuine. If that probability is large, we cannot confidently rule out that it is just random luck. Larger sample sizes make it less likely that a strong observed pattern is a random fluke — this is the same logic as flipping a coin 10,000 times and getting 80% heads (very strong evidence of bias) versus flipping it only 10 times and getting 80% heads (could easily be luck).

Practical takeaway: features with high |r| AND very small p-value are confidently genuine predictors. Features with low |r| and large p-value carry no reliable signal and are not worth including in the model.

PHASE 4: Multicollinearity Check (VIF)

Why This Phase Exists

Phase 3's correlation heatmap gave a visual signal that some features may be highly correlated with each other (e.g., GarageCars and GarageArea). But 'it looked dark red on the heatmap' is not a formal measure. VIF gives a precise number per feature answering: 'how redundant is this feature given all the other features?' This directly relates to Assumption 2 (no/low multicollinearity).

Intuition: Why Multicollinearity Is a Problem

Analogy: asking two experts for a house price estimate when both looked at almost the exact same information (e.g., just the area) makes it hard to decide how much weight to give each opinion. Similarly, when two features carry nearly the same underlying information (GarageCars and GarageArea both essentially measure 'how big is the garage'), the model gets confused about how to split the 'credit' between their coefficients: the split can be arbitrary, small data changes can swing coefficients drastically (instability), and a coefficient's sign can even flip to something counter-intuitive.

Important clarification: multicollinearity does not hurt prediction accuracy much — the model can still make reasonably accurate predictions. The real damage is to INTERPRETABILITY: individual coefficients become unreliable/untrustworthy for explaining 'the effect of this one feature'.

Maths: How VIF Is Calculated

For each feature xi, run a mini regression predicting xi from ALL the other features (xi = a0 + a1x1 + ... + an*xn + error — same idea as normal Linear Regression, but the 'target' here is the feature itself). Compute that regression's R-squared (Ri^2), then:

VIFi = 1 / (1 - Ri^2)

Intuition: if xi cannot be predicted at all from the other features (Ri^2 = 0, fully unique information), VIFi = 1 (best case, no multicollinearity). If xi can be almost perfectly predicted from the other features (Ri^2 close to 1, fully redundant), VIFi approaches infinity (worst case). VIF measures how much a coefficient's variance (uncertainty) is inflated compared to if this feature were completely independent — e.g. VIF = 5 means the coefficient's variance is 5x larger than it would be with zero correlation, making its confidence interval much wider/unreliable.

Thresholds (rule of thumb)

- VIF = 1: no correlation (ideal)
- VIF 1-5: moderate correlation, generally fine
- VIF > 5: concerning, worth attention
- VIF > 10: serious multicollinearity, action needed

What To Do With High VIF Features

- Drop one of the correlated features (keep the more important/complete one)
- Combine them into a single new feature (sum/average) if conceptually similar
- Apply PCA (dimensionality reduction) on the correlated group
- Use Ridge Regression, which handles multicollinearity mathematically without dropping features (a future regularization topic)

Implementation Note

statsmodels.stats.outliers_influence.variance_inflation_factor(exog_array, exog_idx) computes VIF for one column index at a time — loop over all numeric columns. Exclude the target (SalePrice) from this calculation — VIF is only about relationships among predictors, not against the target. Run this on numeric columns only before categorical encoding (Phase 7) is done.

PHASE 5: Categorical Features EDA

Why This Phase Exists

Phases 3-4 only explored numeric features. Ames has 43 categorical columns — ignoring them would mean missing major predictors like Neighborhood (location is one of the biggest price drivers in real estate, and it is a purely categorical feature). This phase is the categorical counterpart of Phase 3: same goal (does this feature genuinely affect the target), different tools since scatter plots/correlation don't work on categories.

Step 1: Boxplots — Category vs SalePrice

How a boxplot is built: sort a group's values and find Q1 (25th percentile), Median/Q2 (50th percentile), Q3 (75th percentile). The box spans Q1 to Q3 (IQR = Q3-Q1, the spread of the middle 50% of data), with

a line at the median. Whiskers extend to the furthest actual data point within Q1-1.5xIQR and Q3+1.5xIQR; points beyond that are plotted as individual outlier dots (the 1.5xIQR convention comes from Tukey — under a normal distribution roughly 0.7% of data falls outside this range).

For a categorical feature (e.g. Neighborhood), one such box is built per category, then all boxes are shown side by side. If boxes sit at clearly different levels (e.g. NoRidge's box much higher than OldTown's), that's a visual signal the category affects price. If boxes mostly overlap, the category may not matter much.

Ordinal vs Nominal note: ordinal categories (ExterQual, KitchenQual: Po<Fa<TA<Gd<Ex) should be plotted in their logical order (not alphabetical) to see the quality-to-price trend clearly. Nominal categories (Neighborhood, SaleType) have no inherent order. This distinction directly feeds into Phase 7 encoding choices: ordinal → OrdinalEncoder (preserves order), nominal → One-Hot/Frequency encoding.

Step 2: ANOVA Test — Formal Confirmation

Core idea: split total variance into BETWEEN-group variance (how far each group's mean is from the overall grand mean) and WITHIN-group variance (how much values vary inside a single group, around its own mean — natural randomness that exists regardless of any real category effect).

F = (variance BETWEEN groups) / (variance WITHIN groups). If F is large, the difference between group means is much bigger than the natural noise within groups — strong evidence the category has a real effect. If F is small (near 1), the between-group difference is no bigger than ordinary within-group randomness — no real category effect, whatever difference appears is likely just chance.

Concrete example computed: OldTown [100,110,90], CollgCr [150,160,140], NoRidge [250,260,240]. Grand mean = 166.67. SS_between = 35,000, SS_within = 600 → F = 175, p-value ~ 0.0000048 → extremely strong evidence Neighborhood genuinely affects price. Contrast: three groups with nearly identical means and similar spread (~150 each) gave F = 0.0, p-value = 1.0 → no real category effect, difference is pure noise.

Why ANOVA instead of many pairwise t-tests: categorical columns often have many groups (Neighborhood has ~25). Running a separate t-test for every pair causes a 'multiple comparisons' problem — enough tests will produce 'significant' results purely by chance (inflated false positive rate). ANOVA compares all groups in a single test, avoiding this issue.

What Insight This Gives, and Where It Is Used

- Feature Engineering: categorical columns with small p-value (genuinely significant, e.g. Neighborhood) should definitely be encoded and included in the model. Columns with large p-value (no real effect) can be deprioritized or dropped, especially if they also have high cardinality (many unique categories) that would otherwise add unnecessary complexity.
- Model Training: gives a prioritization of which features deserve more careful encoding or interaction terms versus which are 'nice to have but not critical'.
- Model Interpretation/Reporting: allows a defensible, evidence-based statement like 'ANOVA confirmed Neighborhood and KitchenQual significantly affect SalePrice ($p < 0.05$), so they were treated as primary predictors' — far more credible than picking features by gut feeling.

Implementation Code

sns.boxplot(x='Neighborhood', y='SalePrice', data=df); plt.xticks(rotation=90). For ANOVA: build one group (Series of SalePrice values) per category using a list comprehension (groups = [df[df['Neighborhood']==cat]['SalePrice'] for cat in df['Neighborhood'].unique()]), then call scipy.stats.f_oneway(*groups) — the * unpacks the list into separate positional arguments since f_oneway expects each group as its own argument, not a single list. Returns (F_statistic, p_value).

PHASE 6: Missing Value Handling

Why This Phase Exists

Phase 1 identified which columns have missing values and what percentage. This phase actually fixes them. Handling this incorrectly (blindly using df.fillna(df.mean()) or df.dropna()) is one of the most destructive mistakes possible before training — it can corrupt real signal in the data.

Two Categories of Missingness

Category A — 'Meaningful NA': the missing value itself is information, not an error. Identified from the Ames data dictionary (data_description.txt). Examples: PoolQC NA = 'No Pool', Alley NA = 'No alley access', FireplaceQu NA = 'No Fireplace', Garage* columns NA = 'No Garage', Bsmt* columns NA = 'No Basement'. These should be filled with the explicit string 'None' (categorical) or 0 (numeric, e.g. GarageArea where no garage exists should be area 0, not a missing value).

Category B — 'Genuinely Missing': data collection gaps where a value should exist but wasn't recorded. Examples: LotFrontage (every house has some frontage, so missing here is a real gap), Electrical (only 1 row missing — a random data-entry gap). Strategy: numeric → median (more robust to outliers/skew than mean); categorical → mode. Better approach for LotFrontage specifically: group-wise median BY Neighborhood, since typical lot size varies a lot by area — an overall median would misrepresent both large-lot and small-lot neighborhoods.

Important Reminder — Train-Test Split and Leakage

Any statistic-based imputation (median/mode) must be FIT only on the train set, then the same fitted value applied to the test set (never recompute a separate median from the test set — that would leak information). 'Meaningful NA' fills ('None'/0) are not affected by this since no statistic is being learned from the data — a fixed label/value is simply being assigned.

Implementation Code

```
cols_none = ['PoolQC','Alley','Fence','FireplaceQu','GarageType','GarageFinish',
             'GarageQual','GarageCond','BsmtQual','BsmtCond','BsmtExposure',
             'BsmtFinType1','BsmtFinType2','MasVnrType']
df[cols_none] = df[cols_none].fillna('None')

cols_zero = ['GarageArea','GarageCars','GarageYrBlt','BsmtFinSF1','BsmtFinSF2',
             'BsmtUnfSF','TotalBsmtSF','MasVnrArea','BsmtFullBath','BsmtHalfBath']
df[cols_zero] = df[cols_zero].fillna(0)

df['LotFrontage'] = df.groupby('Neighborhood')['LotFrontage']\
                      .transform(lambda x: x.fillna(x.median()))

df['Electrical'] = df['Electrical'].fillna(df['Electrical'].mode()[0])
```

```
remaining_missing = df.isnull().sum()
print(remaining_missing[remaining_missing > 0])
```

PHASE 7: Feature Engineering

Why This Phase Exists

EDA and cleaning are done. Now raw columns are transformed so Linear Regression can extract maximum signal. The algorithm itself is fixed — features are the one thing we fully control, which is why 'better features > better algorithm' holds, and why this phase usually improves performance the most.

Step 1: Ordinal Encoding — Quality Columns

Columns like ExterQual, BsmtQual, HeatingQC, KitchenQual, FireplaceQu, GarageQual, GarageCond have a natural order (None < Po < Fa < TA < Gd < Ex). Using OrdinalEncoder with this explicit order preserves the ranking (lost if OneHot/Frequency encoded instead) and gives a directly interpretable coefficient ('one quality point increase raises price by X').

Step 2: New Features (Domain Knowledge)

TotalSF = TotalBsmtSF + 1stFlrSF + 2ndFlrSF; TotalBath = FullBath + 0.5*HalfBath + BsmtFullBath + 0.5*BsmtHalfBath; HouseAge = YrSold - YearBuilt; RemodAge = YrSold - YearRemodAdd. These are simple row-wise arithmetic combinations — safe to compute before the train-test split since no cross-row statistic is involved (result is identical whether computed on the full df or after splitting). Half-bath gets half the weight of a full bath (domain assumption: a half-bath is roughly half as valuable). Raw year columns are dropped after deriving age features since 'how old' is more meaningful to the model than an arbitrary year number, and keeping both would be redundant (multicollinearity).

Step 3: Nominal Categorical Encoding

Remaining categorical columns with no natural order (Neighborhood, SaleType, Foundation, MSZoning, etc.) are split by cardinality, decided on X_train only: low cardinality (<10 unique values) → One-Hot Encoding; high cardinality (e.g. Neighborhood ~25 categories) → Frequency Encoding (avoids an explosion of sparse OHE columns). Frequency map is learned from X_train.value_counts() only, then applied (mapped) to both train and test — an unseen category in test becomes NaN after mapping and is filled with 0 (treated as 'rare/unseen').

Step 4: Skewed Numeric Features — Log Transform

Beyond just the target, every numeric feature's skewness is checked (using scipy.stats.skew) — any feature with |skew| > 0.75 gets np.log1p applied, same logic as the target transform in Phase 2. This helps both Assumption 1 (linearity) and Assumption 4 (normality). The skewed-feature list is decided on X_train only, then the exact same columns are log-transformed in X_test (never re-check skew separately on test — that would risk transforming different columns in train vs test). .clip(lower=0) is applied before log1p as a safety measure since log1p is undefined for negative values.

Step 5: Scaling

StandardScaler on all numeric features. Not mathematically required for the Normal Equation, but critical for Gradient Descent — scaling turns an elongated-ellipse cost function contour into a circular one, giving faster, more stable convergence.

Critical Rule Throughout This Phase

Two kinds of steps: (1) Deterministic/rule-based (new feature formulas, fixed ordinal order, 'None' fills) — safe to do before the split, since no statistic is learned from data and results are identical regardless of split. (2) Statistic-based/learned (medians, frequency maps, skew decision, scaler mean/std, OneHot categories) — must always be FIT on train only, then only TRANSFORMED on test. This train-fit/test-transform pattern is the core principle that prevents data leakage throughout the entire pipeline.

PHASE 9: Scratch Implementation (Normal Equation + Gradient Descent)

Why This Phase Exists

With X_train_final, X_test_final, y_train_log, y_test_log ready, Linear Regression is implemented from scratch two ways — Normal Equation (direct formula) and Gradient Descent (iterative) — to understand the mechanics before relying on sklearn. Both should give approximately the same answer; they differ only in method. A bias/intercept column of all 1s must be added to X first, since the model form is y_hat = w0 + w1x1 + ... and w0 needs a corresponding constant feature in matrix form.

Method 1: Normal Equation

w = (X^T X)^-1 X^T y — computed directly via matrix operations.

np.linalg.pinv() (pseudo-inverse) is used instead of np.linalg.inv(), because inv() only works for a perfectly invertible matrix. Even after reducing multicollinearity with VIF, some residual correlation may remain, making X^T X near-singular — pinv() always returns a stable solution.

Method 2: Gradient Descent

Cost: J(w) = (1/2m) * sum((y_i - y_hat_i)^2). Update rule: w := w - alpha * (1/m) * X^T(Xw - y). Weights start at zero, then each iteration computes predictions, the error, the gradient (direction to reduce cost), and takes a small step opposite to the gradient. Cost is tracked every iteration to plot a convergence curve — it should decrease smoothly and flatten; oscillation or increase signals too high a learning rate, and very slow decrease signals too low a learning rate or too few iterations.

Both methods' resulting weights should closely match — this cross-check confirms the scratch implementation is mathematically correct.

```
X_train_b = np.c_[np.ones((X_train_dense.shape[0], 1)), X_train_dense]
X_test_b = np.c_[np.ones((X_test_dense.shape[0], 1)), X_test_dense]

def normal_equation(X, y):
    XtX = X.T @ X
    XtX_inv = np.linalg.pinv(XtX)
    Xty = X.T @ y
    return XtX_inv @ Xty

w_normal_eq = normal_equation(X_train_b, y_train_arr)
```

```
def gradient_descent(X, y, learning_rate=0.01, n_iterations=1000):
    m, n = X.shape
    w = np.zeros(n)
    cost_history = []
    for i in range(n_iterations):
        y_pred = X @ w
        error = y_pred - y
        gradient = (1/m) * (X.T @ error)
        w = w - learning_rate * gradient
        cost = (1/(2*m)) * np.sum(error**2)
        cost_history.append(cost)
    return w, cost_history

w_gd, cost_history = gradient_descent(X_train_b, y_train_arr, learning_rate=0.01, n_iteration
s=1000)
```

PHASE 10: sklearn Implementation

Why This Phase Exists

Verify the scratch implementation against sklearn's LinearRegression on the same data, and see the production-grade equivalent (real projects use libraries, not scratch code). Note: sklearn's LinearRegression handles the intercept internally (fit_intercept=True by default), so it must be given X_train_dense (without the manually added ones-column) — not X_train_b — otherwise the intercept would be double-counted.

All three methods' predictions (Normal Equation, Gradient Descent, sklearn) and their coefficients/intercepts should be very close to each other — this cross-check confirms mathematical correctness of the scratch code.

```
from sklearn.linear_model import LinearRegression

sklearn_model = LinearRegression()
sklearn_model.fit(X_train_dense, y_train_arr)

y_pred_normal_eq = X_test_b @ w_normal_eq
y_pred_gd = X_test_b @ w_gd
y_pred_sklearn = sklearn_model.predict(X_test_dense)
```

PHASE 11: Evaluation + Statistical Inference

Why This Phase Exists

Compute prediction metrics on the original rupee scale (converting log predictions back with np.expm1, the exact inverse of np.log1p used in Phase 2), check residual diagnostics for the Linear Regression assumptions, and run formal statistical inference (F-test, t-test) via statsmodels.

Metrics: R^2, Adjusted R^2, RMSE, MAE

Computed on y_test_original vs each model's predictions after expm1 conversion. All three implementations (Normal Equation, Gradient Descent, sklearn) should give nearly identical R^2/RMSE/MAE — small differences from Gradient Descent are expected if convergence wasn't perfectly

complete.

Residual Diagnostics (Assumption Checks)

- Residuals vs Predicted plot: checks Assumption 3 (homoscedasticity) — points should be randomly scattered around 0 with no funnel/cone shape.
- Q-Q Plot of residuals: checks Assumption 4 (normality) — points should follow the diagonal line; curving away at the tails indicates non-normal residuals.
- Residuals histogram: should look roughly bell-shaped/symmetric.

Breusch-Pagan Test — Formal Heteroscedasticity Check

H0: homoscedasticity holds (constant residual variance). If p-value < 0.05, H0 is rejected — heteroscedasticity is present (Assumption 3 violated), suggesting a target transformation (already applied via log) or weighted least squares.

Statistical Inference — statsmodels OLS Summary

sm.OLS(y_train, sm.add_constant(X_train)).fit().summary() gives: R-squared/Adj. R-squared (cross-check against manual calculation); F-statistic and its p-value, testing H0 = 'all coefficients are zero' (a very small p-value rejects H0, confirming the overall model is statistically meaningful); and a per-feature coefficient table with a t-test for each (H0 = 'this coefficient is zero' — a p-value < 0.05 means that specific feature's effect is statistically significant; features with high p-value aren't reliably distinguishable from having zero effect, regardless of the coefficient's raw magnitude).

```
y_test_original = np.expm1(y_test_arr)
y_pred_sklearn_original = np.expm1(y_pred_sklearn)

def evaluate_model(y_true, y_pred, n_features, model_name):
    n = len(y_true)
    ss_tot = np.sum((y_true - np.mean(y_true))**2)
    ss_res = np.sum((y_true - y_pred)**2)
    r2 = 1 - (ss_res / ss_tot)
    adj_r2 = 1 - (1 - r2) * (n - 1) / (n - n_features - 1)
    rmse = np.sqrt(np.mean((y_true - y_pred)**2))
    mae = np.mean(np.abs(y_true - y_pred))
    return {'model': model_name, 'r2': r2, 'adj_r2': adj_r2, 'rmse': rmse, 'mae': mae}

import statsmodels.api as sm
from statsmodels.stats.diagnostic import het_breuschpagan

X_train_with_const = sm.add_constant(X_train_dense)
ols_model = sm.OLS(y_train_arr, X_train_with_const).fit()
print(ols_model.summary())

bp_test = het_breuschpagan(ols_model.resid, ols_model.model.exog)
```

Doubts Clarified During Phase 1 Discussion

Q: Is this dataset definitely a Regression problem?

Yes. The problem type (Regression vs Classification vs Unsupervised) is decided purely by the TARGET variable, not by the features. SalePrice is continuous/numeric (min 34,900 to max 755,000, effectively infinite possible values) → Regression problem.

Decision rule: Target continuous (price, temperature, quantity) = Regression. Target discrete/categorical (spam/not-spam, class A/B/C) = Classification.

Quick check used: df['SalePrice'].nunique() (should be large) and df['SalePrice'].dtype (should be int64/float64).

Q: But most of the features are categorical — does that change the problem type?

No. Feature types are irrelevant to deciding Regression vs Classification. Features are just inputs — whether numeric or categorical, the model still learns a relationship toward the target. Feature types only affect HOW we do EDA (numeric → scatter/correlation, categorical → boxplot/ANOVA) and HOW we do FE (numeric → scaling/log-transform, categorical → encoding). The problem type is decided only by the target variable.

Analogy: predicting income (continuous) from age and city (mixed feature types) is Regression. Predicting churn (Yes/No) from the exact same features becomes Classification. Same features, different problem type — because the target changed.

Ames Housing dataset composition:

35 numeric (int64) + 3 float64 + 43 categorical (object/str) columns = 81 total (before dropping Id). Target: SalePrice (continuous).